7.1 Theory: Kernel Methods & Neural Networks

a) Valid Kernel Function and Mercer's Theorem

Valid Kernel Function

Imagine you have data points that are not linearly separable in their original space. A kernel function $K(\mathbf{x}, \mathbf{z})$ is a clever shortcut instead of manually transforming every data point into a higher-dimensional space and then computing dot products, the kernel does both steps at once, implicitly.

Formally, we say K is a valid kernel if there exists some feature map ϕ such that:

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{z})$$

You never actually compute ϕ the kernel gives you the answer directly. For K to be a valid kernel, the matrix of all pairwise values $G_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ (called the **Gram matrix**) must be symmetric and positive semi-definite (all eigenvalues ≥ 0).

A common example is the RBF (Gaussian) kernel:

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right)$$

This single formula secretly computes a dot product in an *infinite-dimensional* space something you could never do explicitly.

Mercer's Theorem

Mercer's theorem tells us exactly when a kernel is valid:

A function $K(\mathbf{x}, \mathbf{z})$ is a valid kernel if and only if, for any finite set of points, its Gram matrix is symmetric and positive semi-definite.

The practical takeaway: you do not need to find ϕ and prove it works. Just check that the Gram matrix satisfies this condition, and Mercer guarantees that some valid ϕ exists behind the scenes.

b) Neural Network Hidden Layer as $\phi(\mathbf{x})$

The hidden layer is a learned feature map

Think of a neural network as two parts working together. The hidden layers take the raw input $\mathbf{x}$ and transform it into something more useful. The final layer then does something simple just a linear operation on top of that transformation.

We can write the hidden-layer output as $\phi(\mathbf{x})$:

$$\mathbf{x} \xrightarrow{\text{hidden layers (learn useful features)}} \phi(\mathbf{x}) \xrightarrow{\text{linear output layer}} \hat{y}$$

So the network is really doing two things: (1) find a good way to represent the data, and (2) fit a line through that representation. The hidden layers handle the hard part of finding $\phi(\mathbf{x})$ automatically.

How this connects to the kernel trick

Once you have $\phi(\mathbf{x})$ from a neural network, you can define a kernel:

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{z})$$

This is exactly what kernel methods do compute similarity between points via a dot product in feature space. The big difference is *who chooses* ϕ:

	Kernel Methods (e.g. SVM)	**Neural Networks**
Who picks ϕ?	You pick it upfront (RBF, poly, ...)	Network learns it from data
Feature space size	Can be infinite	Fixed (hidden layer width)
How similarity is found	$K(\mathbf{x}, \mathbf{z})$ formula	Dot product of activations

In short: kernel methods and neural networks are solving the same underlying problem — find a good feature space so that a linear model works well. Kernel methods ask you to guess the right feature space upfront by picking a kernel. Neural networks skip that guess and just learn the feature space directly from the data.